Assignment - 4

Optimizing Transformer Translation with Ray Tune & Optuna

Submission Requirements

Please submit the link to your GitHub repository containing the following:

- **rollno_ass_4_tuned_en_to_hi.ipynb/*.py**: Your modified Jupyter Notebook containing the Ray Tune implementation.
- **rollno_ass_4_report.pdf**: A brief 1-2 page report detailing:
  - The baseline metrics (Time, Final Loss, BLEU Score at 100 epochs).
  - The 4+ hyperparameters you chose to tune, along with the ranges you provided.
  - The **Best Configuration** found by you.
  - The final metrics of your best model (Time, Final Loss, BLEU Score) and the number of epochs it took to match/beat the baseline.
- **rollno_ass_4_best_model.pth**: The saved weights of your best-performing model from the tuning sweep.

Grading Rubric

- **Baseline Execution (10%)**: Correctly identified and documented the baseline metrics.
- **Code Refactoring (20%)**: Successfully adapted the PyTorch training loop to interface with Ray Tune.
- **Hyperparameter Setup & Search (30%)**: Correctly configured at least 4 hyperparameters and implemented OptunaSearch.
- **Efficiency Goal (20%)**: Matched Baseline (achieved in en_to_hi.ipynb) or exceeded the BLEU (0.50) score using ≤ X epochs.
- **Report (20%)**: Clear, concise comparison and documentation of findings.

Overview

In this assignment, you will be working with a custom PyTorch Transformer model designed to translate text from English to Hindi. The provided Jupyter Notebook (en_to_hi.ipynb) contains a complete pipeline: data preprocessing, vocabulary building, a from-scratch Transformer architecture, and a training loop.

Currently, the model trains for 100 epochs with a hardcoded set of hyperparameters. While it achieves a

baseline translation quality, it is computationally expensive and likely suboptimal.

Your mandate is as follows:

1. Establish a baseline performance by training the model as-is. **Note: Everything you work on (code, notebooks, saved models, and reports) needs to be pushed to your GitHub repository.**
2. Refactor the code to use **Ray Tune** paired with the **Optuna** search algorithm to find a better hyperparameter configuration. Your ultimate goal is to **match or exceed the baseline BLEU score in significantly fewer epochs** (e.g., 20-X epochs instead of 100; X<100).

Link: https://drive.google.com/drive/folders/19fhUruT03NkYp6u0uax-zjaZ6dZHUjzz?usp=sharing

Part 1: Establish the Baseline

Before optimizing, you must know what you are comparing against.

1. **Run the Notebook:** Execute the provided en_to_hi.ipynb notebook from start to finish without making any architectural or hyperparameter changes.
2. **Record Metrics:**
   - Note the total time it takes to complete the 100 epochs.
   - Record the final training loss.
   - Run the NLTK evaluation cell at the end of the notebook and record the final **BLEU Score**.
3. **Save Baseline Weights:** Keep the transformer_translation_final.pth file generated at the end of the run for your records.

Part 2: Refactoring for Ray Tune & Optuna

To optimize our model, we will use **Ray Tune** for scalable hyperparameter tuning and **Optuna** as the underlying search algorithm to intelligently navigate the search space.

Step 2.1: Modify the Training Loop

You will need to adapt the existing train() function so that Ray Tune can manage it.

- Create a new function, e.g., train_tune(config), that accepts a config dictionary.
- Inside this function, initialize the Transformer model, optimizer, and criterion using values from the config dictionary rather than hardcoded global variables.
- **Important:** Inside your epoch loop, replace the standard printing/saving logic with Ray's reporting mechanism: ray.train.report({"loss": epoch_loss).

Step 2.2: Define the Search Space

You must vary at least **4 different hyperparameters**. For a Transformer model, some of the most impactful parameters include:

1. **Learning Rate (lr)**: Try searching on a logarithmic scale (e.g., 1e-5 to 1e-3).
2. **Batch Size (batch_size)**: e.g., 16, 32, 64.
3. **Number of Attention Heads (num_heads)**: e.g., 4, 8. *(Note: Ensure D_MODEL is always divisible by num_heads)*.

4. **FeedForward Dimension (d_ff)**: e.g., 1024, 2048.
5. **Dropout Rate (dropout)**: e.g., 0.1 to 0.4.
    - *Use Ray Tune's search space APIs like tune.loguniform(), tune.choice(), and tune.uniform().*

Step 2.3: Configure Optuna and Run the Sweep

Set up the Tuner to utilize Optuna.

```
from ray import tune
from ray.tune.search.optuna import OptunaSearch

# Define your Optuna search algorithm
optuna_search = OptunaSearch(metric="loss", mode="min")

# Initialize the Tuner
tuner = tune.Tuner(
    train_tune,
    tune_config=tune.TuneConfig(
        search_alg=optuna_search,
        num_samples=20, # Number of different hyperparameter combinations to try
    ),
    param_space=your_config_dict
)

results = tuner.fit()
```

Part 3: The Efficiency Challenge

Training a Transformer for 100 epochs takes a long time. By finding the optimal combination of learning rate, batch size, and regularization (dropout), the model can converge much faster.

Your Goal: Achieve the same (or better) BLEU score as your Part 1 baseline, but your Ray Tune trials should be capped at **no more than X epochs** per trial.

Hint: To make your search even more efficient, look into using the **ASHA Scheduler** (ray.tune.schedulers.ASHAScheduler) alongside Optuna. ASHA will automatically detect and terminate underperforming trials early, saving significant compute time.